
Projet Corner Detection

-

Rapport de validation

Glen COZIC
2 octobre 2026

Table des matières

1	Affichage sur l'écran d'une image	3
1.1	[PL-IP-001] Source Vidéo TPG [1] et [PL-DISP-002] Affichage écran VGA [1]	3
1.1.1	Horloge	3
1.1.2	Affichage sans AXI	3
1.1.3	IP AXI stream Master	4
1.1.4	IP AXI stream Slave	4
1.1.5	Affichage avec AXI	4
1.2	[PL-IP-000] Source Vidéo mémoire [1]	8
1.3	[PS-001] Mise en mémoire manuelle pattern [1]	10
1.4	[PL-IP-002] Sélection de source vidéo init [1]	10
1.4.1	AXISelect	10
1.4.2	Commandes	11
2	Traitement d'image	11
2.1	RGBtoGray	11
2.2	Recadrage	12
2.3	[PL-IP-006] Filtre gaussien [1]	12
2.4	[PL-IP-007] Filtre Harris [1]	12
2.5	[PL-IP-008] Filtre Harris binarisation [1]	12
2.6	[PL-IP-009] Filtre Harris seuillage paramétrable [1]	12
2.7	[PL-DISP-001] Overlay [1]	12
2.8	Test complet	12
3	Nice to have	13
3.1	[PL-IP-003] Sélection de source vidéo à chaud [2]	13
3.2	[PL-DISP-003] Mode d'affichage [2]	13
3.3	[PL-IP-010] Filtre Harris retour de données [2]	13
3.4	[PS-003] Retour de données UART [2]	13
3.5	[PS-002] Mise en mémoire automatique pattern [2]	13

Table des figures

1	Simulation de la PLL	3
2	TPG et ControleurVGA	3
3	AXI4SM_sim1	4
4	AXI4SM_sim2	4
5	TPG vers AXI vers Contrôleur VGA 2	5
6	TPG vers AXI vers Controleur VGA simulation 1	5
7	TPG vers AXI vers Controleur VGA simulation 2	6
8	TPG vers AXI vers Controleur VGA simulation 3	6
9	TPG vers AXI vers Controleur VGA simulation 4	7
10	TPG vers AXI vers Controleur VGA simulation 5	7
11	DMA vers le Contrôleur VGA	8
12	Image correcte	9
13	Image décalée	9
14	AXISelect	10
15	IP Commandes	11
16	Simulation du RGBtoGray	11

1 Affichage sur l'écran d'une image

1.1 [PL-IP-001] Source Vidéo TPG [1] et [PL-DISP-002] Affichage écran VGA [1]

1.1.1 Horloge

Le contrôleur VGA nécessite une horloge à 25MHz. Le choix est fait d'utiliser cette même horloge à 25MHz pour toute l'architecture. Une PLL génère l'horloge de 25MHz à partir d'une à 125MHz.

- Rédaction d'un testbench pour la PLL
- Vérification de la fréquence à 25MHz en sortie de la PLL

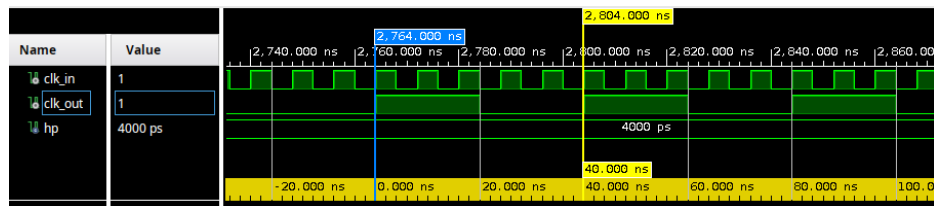


FIG. 1 – Simulation de la PLL

1.1.2 Affichage sans AXI

L'affichage sans AXI est vérifié avec l'architecture suivante :

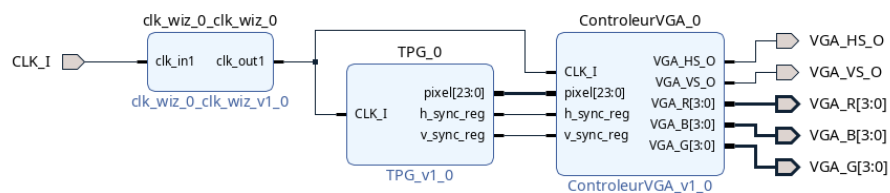


FIG. 2 – TPG et ControleurVGA

Pas de liaison AXI4 stream mais une liaison logique simple entre les 2 bloc. Le clk_wiz génère une horloge de 25MHz avec une horloge de 125MHz.

- L'écran affiche le flux 640x480 à 60Hz.
- La mir RBG dynamique est affichée sans décalage ni déformation.

1.1.3 IP AXI stream Master

Création de l'IP AXI4_stream_Master et d'un testbench correspondant.

Le fonctionnement de l'IP est vérifié en simulation :

- vérification du stockage dans la FIFO des donnée data_in
- vérification du handshake entre tvalid et tready en sortie de l'IP
- vérification de la temporisation de tdata si tready n'est pas à 1
- vérification du parcours complet de l'image avec h_count et v_count
- vérification de la levée de tlast à la fin de chaque ligne
- vérification de la levée de tuser à la fin de chaque image

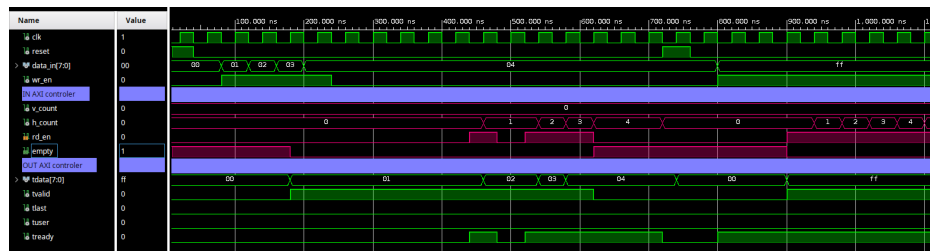


FIG. 3 – AXI4SM_sim1

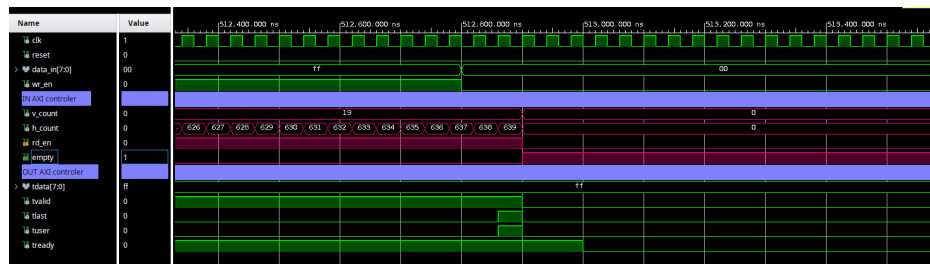


FIG. 4 – AXI4SM_sim2

Déclinaison de l'IP AXI4_stream_Master en 3 IP pour des transfert de 1b, 8b et 24b.

1.1.4 IP AXI stream Slave

Création de l'IP AXI_stream_Slave avec un paramètre générique sur la taille des mots transmis.

1.1.5 Affichage avec AXI

Modification des IP TPG et ControleurVGA pour générer la synchronisation. À présent les signaux de synchronisation h_sync et v_sync sont entièrement gérés par

le controleur VGA.

La transmission par AXI4 permet de transmettre tlast (fin de ligne) et tuser (fin d'image). À partir de ces 2 signaux, l'IP contrôleur VGA pilote les 2 signaux de synchronisation.

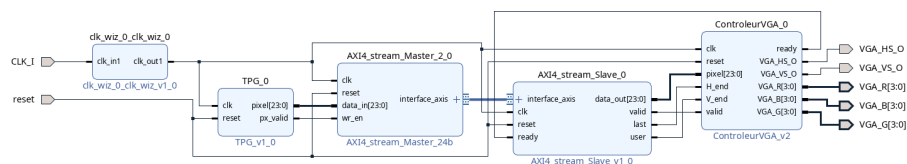


FIG. 5 – TPG vers AXI vers Contrôleur VGA 2

Le fonctionnement est vérifié en simulation.

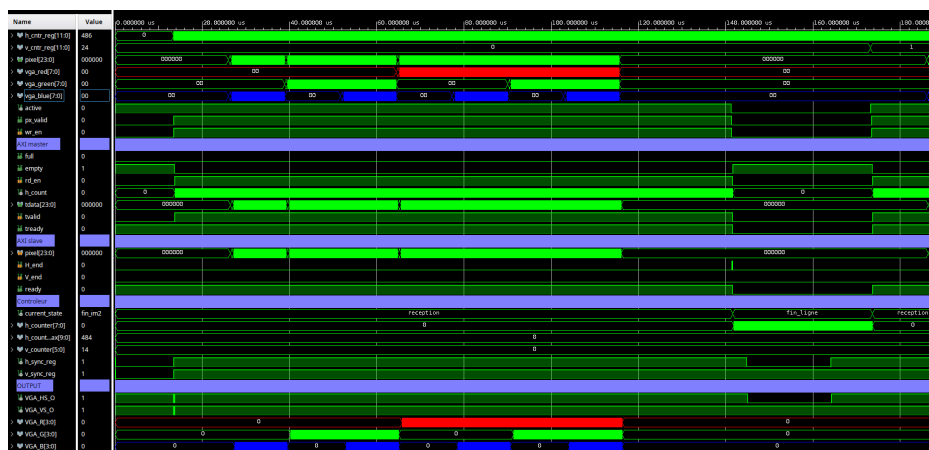


FIG. 6 – TPG vers AXI vers Contrôleur VGA simulation 1

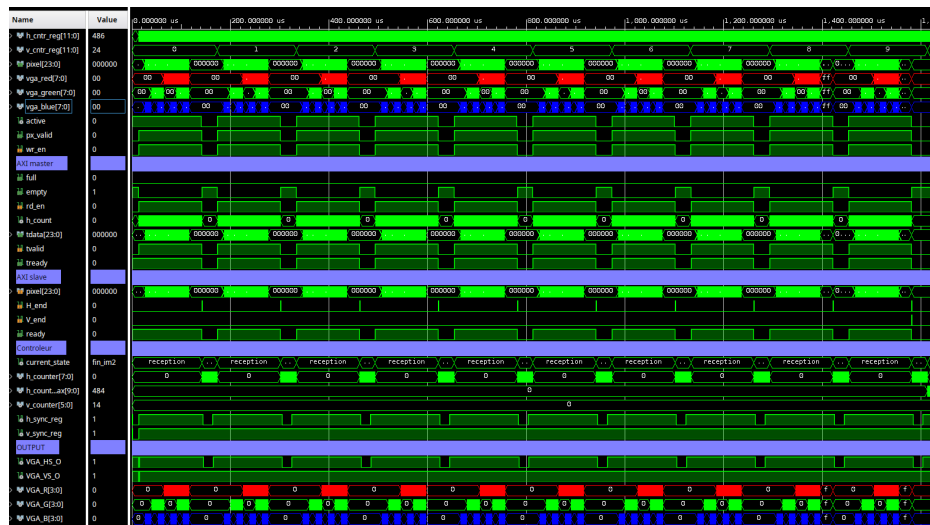


FIG. 7 – TPG vers AXI vers Controleur VGA simulation 2

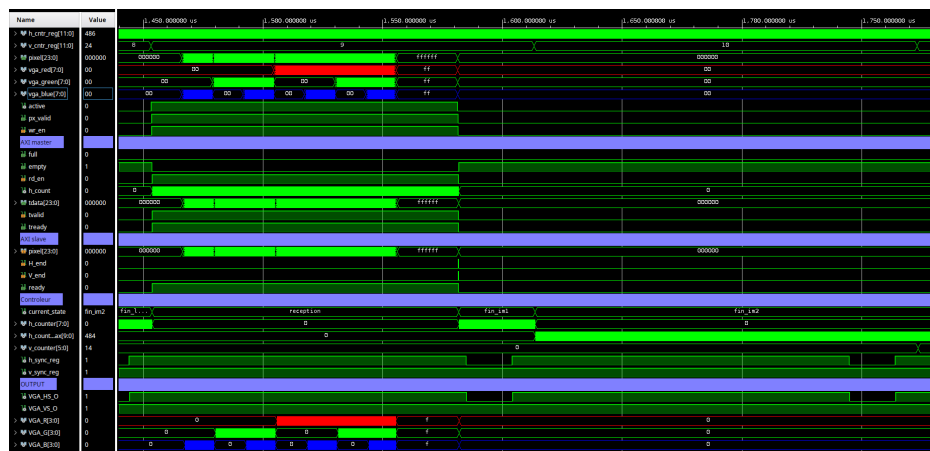
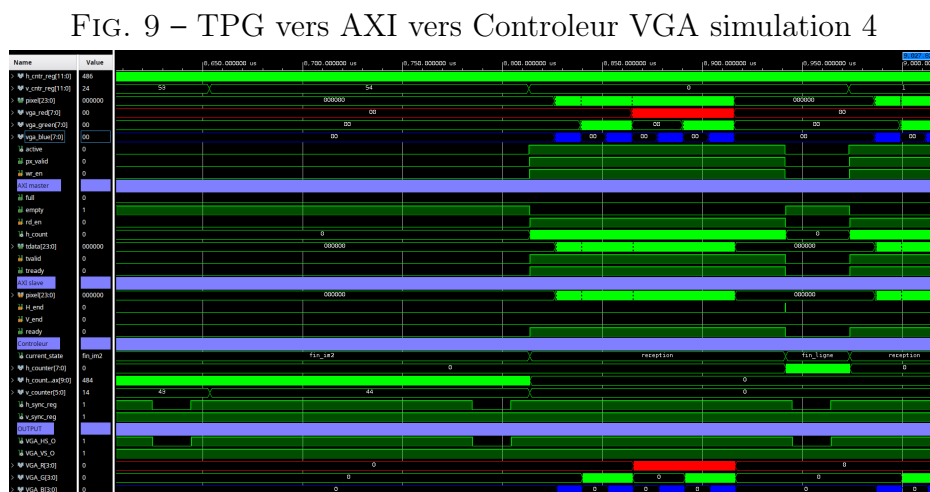


FIG. 8 – TPG vers AXI vers Controleur VGA simulation 3



- les données en sortie du TPG sont identiques aux données de sortie du ContrôleurVGA
- Les compteurs de pixels et de lignes commencent et se réinitialisent correctement
- Les signaux de synchronisation (VGA_HS_O et VGA_VS_O) sont identiques à ceux de "Cora-Z7-07S-Pmod-VGA-hw"

Démonstration sur l'écran du fonctionnement :

- L'écran affiche le flux 640x480 à 60Hz.
- La mir RBG dynamique est affichée sans décalage ni déformation.

1.2 [PL-IP-000] Source Vidéo mémoire [1]

Récupération de l'architecture Référence design.

Intégration du DMA à l'architecture : Les ports `image_w` et `image_h` prennent des constantes correspondants aux dimensions de l'image (640 x 480). Le port `ap_start` est fixé à 1. Le port `image_in` prend l'adresse de base de la DDR, avec un offset pour le code de l'application, c'est-à-dire 0x00400000.

Gestion des reset : le DMA et le Zynq nécessitent une reset actif à l'état bas. Les IP AXI stream et le contrôleur VGA ont un reset actif à l'état haut. Le reset est placé sur un des bouton de la carte.

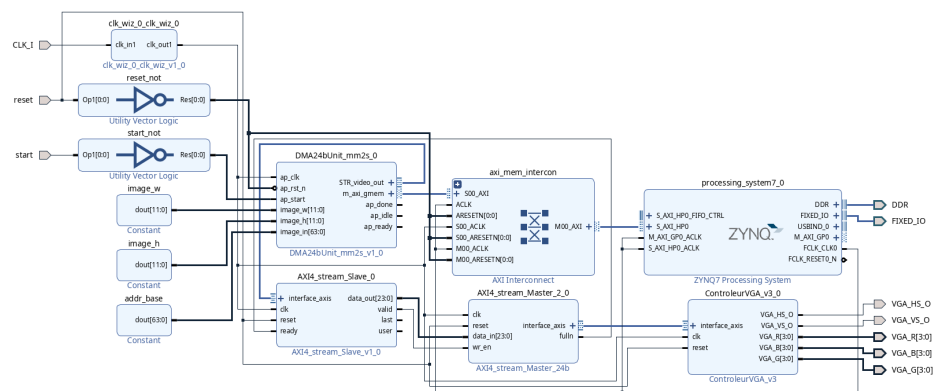


FIG. 11 – DMA vers le Contrôleur VGA

Le DMA lit dans la DDR à l'adresse indiquée et l'écran affiche l'image. Cependant l'image est décalée. Le début de l'image ne correspond pas au début de l'écran. L'arrivée du premier pixel n'est pas synchronisé avec le début de la trame VGA. L'image apparaît donc avec un décalage. Celui-ci n'est pas régulier et change d'une démonstration à l'autre. Il arrive parfois que ce décalage soit nul et que l'image s'affiche correctement.

La gestion du signal `tready` du contrôleurVGA est modifiée mais cette solution ne semble pas régler le problème complètement.



FIG. 12 – Image correcte



FIG. 13 – Image décalée

Le DMA est très difficile à simuler. Le seul moyen d'observer les signaux est en utilisant une ILA. Mais cela rends le débogage lent et fastidieux.

1.3 [PS-001] Mise en mémoire manuelle pattern [1]

La mise en mémoire manuelle de l'image fonctionne. Dans la console XSCT de Vitis, il est possible de charger une image binaire grâce à la commande

```
mwr -bin -file <chemin>/sample\_in\_rgb.bin 0x00400000 230400
```

Après le chemin de l'image, il faut renseigner l'adresse d'écriture dans la DDR et le nombre de mots de 32bits à écrire. Ici l'image fait 640x480 pixels en 3 canaux de couleur codés sur 8 bits. L'image complète est codée entièrement sur 640x480x3x8=7 372 800bits soit 230 400 mots de 32 bits.

- Récupération de l'image depuis la mémoire avec la commande `mrd -bin -file`
- Vérification avec ImageJ que l'image récupérée est identique à celle mise en mémoire

1.4 [PL-IP-002] Sélection de source vidéo init [1]

1.4.1 AXISelect

L'IP AXISelect est très simple et donc est validé directement en démonstration sans simulation.

L'AXISselect permet de choisir lors de l'initialisation la source vidéo. Le signal de commande du AXISselect est placé sur un bouton de la carte. Cela permet aussi de choisir à chaud la source vidéo.

L'architecture suivante intègre le TPG et le DMA connectés à l'AXISelect. Elle sert à la démonstration de l'AXISelect.

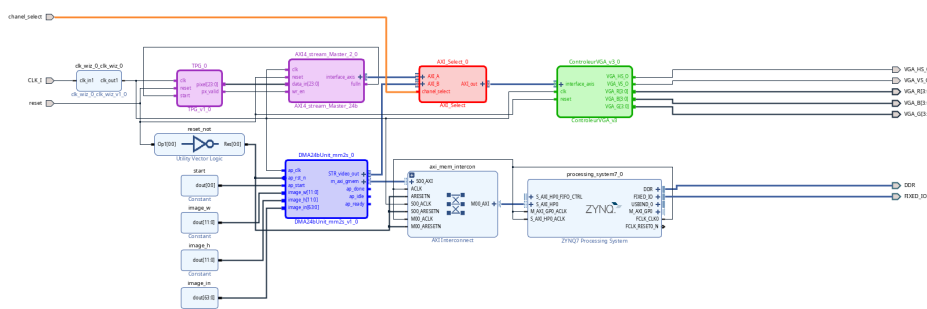


FIG. 14 – AXISelect

À l'appui du bouton, l'écran change d'affichage et passe du TPG au DMA.

1.4.2 Commandes

L'IP Commandes reçoit par une connexion AXI4 Lite une commande pour les AXISelect. Le choix de la source vidéo est donc fait au niveau du PS de manière automatique.

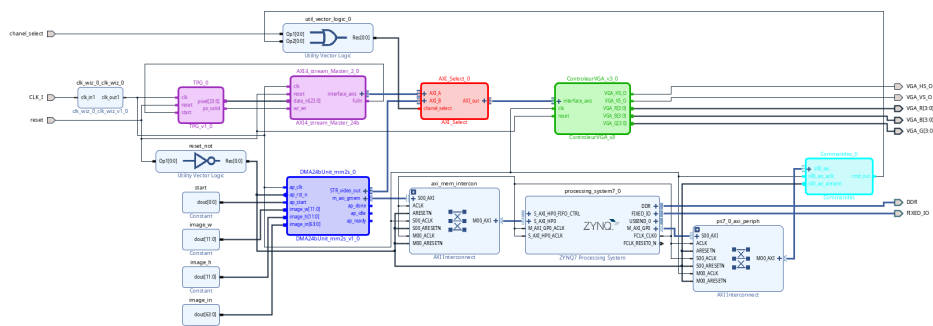


FIG. 15 – IP Commandes

Le fonctionnement des IP Commandes et AXISelect est validé en démonstration avec une programme simple sur Vitis. Ce dernier alterne les sources vidéo toute les secondes environ.

2 Traitement d'image

2.1 RGBtoGray

Tests unitaires :

- En simulation, différentes données sont placées en entrée.
- Le resultat en sortie correspond à la moyennes des cannaux RGB d'entrée.
- En simulation, traitement d'une petite mir RGB.

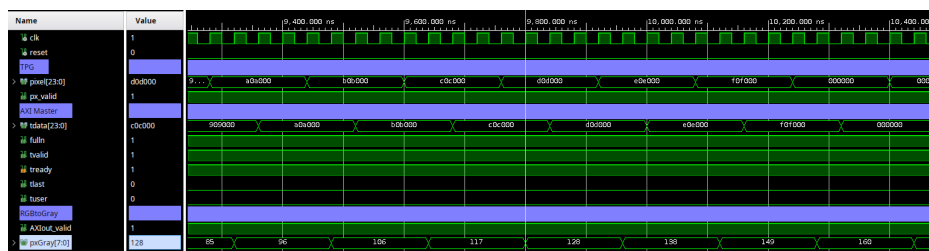


FIG. 16 – Simulation du RGBtoGray

Ici, le TPG envoie les pixels de la mir à un bloc AXI4_stream_Master qui lui même les envoie au RGBtoGray. Pour vérifier le calcul de moyenne, on prends par exemple la donnée envoyée par le TPG à 9 600ns : 0xC0C000. Le bloc AXI4_stream_Master reçoit et envoie cette donnée au RGBtoGray. La donnée de sortie est 128. La donnée 0xC0C000 correspond au trois canaux RGB : 0xC0, 0xC0 et 0x00 soit en décimal : 192, 192 et 0. La moyenne est donc $(192+192+0)/3 = 128$. Le calcul rendu par l'IP RGBtoGray est correct.

La démonstration n'est pas mise en œuvre par manque de temps.

2.2 Recadrage

Non mis en œuvre par manque de temps.

2.3 [PL-IP-006] Filtre gaussien [1]

Non mis en œuvre par manque de temps.

2.4 [PL-IP-007] Filtre Harris [1]

Non mis en œuvre par manque de temps.

2.5 [PL-IP-008] Filtre Harris binarisation [1]

Non mis en œuvre par manque de temps.

2.6 [PL-IP-009] Filtre Harris seuillage paramétrable [1]

Non mis en œuvre par manque de temps.

2.7 [PL-DISP-001] Overlay [1]

Non mis en œuvre par manque de temps.

2.8 Test complet

Non mis en œuvre par manque de temps.

3 Nice to have

3.1 [PL-IP-003] Sélection de source vidéo à chaud [2]

La sélection de la source est faite à chaud grâce aux IPs Commandes et AXISelect. La source alterne toute les secondes.

3.2 [PL-DISP-003] Mode d'affichage [2]

Non mis en œuvre par manque de temps.

3.3 [PL-IP-010] Filtre Harris retour de données [2]

Non mis en œuvre par manque de temps.

3.4 [PS-003] Retour de données UART [2]

Non mis en œuvre par manque de temps.

3.5 [PS-002] Mise en mémoire automatique pattern [2]

Non mis en œuvre par manque de temps.